

SKAN.AI

Agentic AI Terminology

A Framework for Standardization of
Agentic AI Terminology

Published by Skan, Inc. · skan.ai

Companion to the SOA-to-Agentic AI Terminology Mapping ·

github.com/Skan-dot-ai/soa-to-agentic-terms

Licensed CC-BY 4.0. Attribution to Skan, Inc. required for reuse.

The current alphabet soup challenge in Agentic AI needs a fix

The enterprise AI landscape faces a critical challenge: the absence of standardized terminology and architectural frameworks for Agentic AI systems. Unlike established enterprise technologies that have evolved consistent vocabularies over decades, Agentic AI operates in a linguistic wilderness where fundamental concepts lack universal definitions. Ask three practitioners to define an "AI agent" and you'll likely receive four different answers — ranging from simple chatbots to complex autonomous systems capable of multi-step reasoning and decision-making.

This definitional chaos extends beyond academic debate. It creates real barriers to enterprise adoption, procurement decisions, and system integration. CIOs struggle to evaluate vendors when each uses proprietary terminology. Development teams face integration challenges when components described as "agents" operate under vastly different paradigms. Risk management becomes nearly impossible when the scope and capabilities of "agentic systems" remain undefined.

The consequences ripple through the entire enterprise AI ecosystem. Investment decisions suffer from unclear value propositions. Technical architectures become fragmented as teams interpret concepts differently, leading to inconsistencies. Most critically, the promise of autonomous business process automation remains unrealized due to the inability to establish consistent, reliable frameworks for deployment and governance.

Lessons from Service-Oriented Architecture: a proven foundation

Service-Oriented Architecture emerged in the early 2000s as a response to similar challenges in enterprise system integration. Before the SOA era, organizations struggled with monolithic applications that were difficult to maintain, scale, and integrate. The introduction of standardized concepts — such as services, contracts, registries, and orchestration — transformed how enterprises approached system architecture.

SOA's success stemmed not just from its technical merits, but from its establishment of a common vocabulary and architectural patterns. Terms like "service contract," "loose coupling," and "service composition" became universally understood, enabling organizations to design, implement, and maintain complex distributed systems with confidence. The emergence of standards like SOAP, WSDL, and BPEL provided concrete implementation frameworks that translated architectural concepts into actionable technologies.

The parallels to today's Agentic AI challenges are striking. Both domains involve autonomous entities (services vs. agents) that must collaborate to achieve business objectives. Both require orchestration mechanisms to coordinate complex workflows. Both demand governance frameworks to ensure reliability, security, and compliance. Most importantly, both benefit from standardized approaches that reduce implementation risk and accelerate adoption.

However, Agentic AI introduces capabilities that extend far beyond traditional SOA concepts. While services execute predefined operations, agents can adapt, learn, and make autonomous decisions. This evolution doesn't invalidate SOA principles — instead, it suggests that SOA's architectural wisdom can serve as a foundation for more sophisticated agentic paradigms.

(Remember, we are not claiming SOA as our own, just an inspiration to come up with Agentic AI terms as a common standard for the benefit of all stakeholders.)

Avoiding the pitfalls: learning from SOA's limitations

SOA's journey provides valuable cautionary lessons for the Agentic AI community. Despite its conceptual elegance, SOA implementations often suffered from over-engineering, vendor lock-in, and performance bottlenecks. The promise of seamless service composition frequently collided with the reality of complex integration challenges and governance overhead.

Complexity creep. SOA architectures often became unwieldy as organizations created increasingly granular services. The overhead of service interactions sometimes exceeded the benefits of modularization. Agentic AI risks similar complexity if agents are designed too narrowly or if coordination mechanisms become overly sophisticated.

Vendor fragmentation. Despite efforts to standardize, SOA implementations varied significantly across vendors, creating practical interoperability challenges. The Agentic AI space already shows signs of similar fragmentation, with proprietary agent frameworks and incompatible orchestration approaches.

Performance penalties. Service-oriented systems often introduce latency and resource overhead, making them unsuitable for high-performance scenarios. Agentic systems face even greater performance challenges due to the computational demands of AI inference and decision-making.

Governance gaps. Many SOA implementations lacked robust governance frameworks, leading to service sprawl, inconsistent quality, and operational

challenges. Agentic AI systems require even more sophisticated governance due to their autonomous nature and potential for unexpected behaviors.

The key lesson from SOA's experience is that architectural frameworks must strike a balance between standardization and flexibility, providing clear guidelines while avoiding overprescription that stifles innovation.

The need for agentic architecture standards

The current Agentic AI landscape resembles the pre-SOA era of enterprise computing — fragmented, proprietary, and lacking cohesive architectural principles. While individual vendors have developed impressive capabilities, the absence of standardized frameworks limits broader adoption and integration.

What the industry needs is not just terminology mapping, but comprehensive architectural standards that address several challenges and considerations. (The terminology mapping is just a starting point in a long-term endeavor.)

Agent definition and classification. Clear taxonomies that distinguish between different types of agents based on autonomy levels, decision-making capabilities, and interaction patterns. This enables organizations to select appropriate agent types for specific use cases and establish consistent expectations for behavior and performance.

Interaction protocols. Standardized methods for agents to communicate, negotiate, and collaborate. Unlike simple service-to-service communication, agent interactions may involve complex negotiations, shared learning, and dynamic role allocation.

Orchestration patterns. Frameworks for coordinating multi-agent workflows that can adapt to changing conditions and handle exceptions gracefully. These patterns must account for agent autonomy while maintaining overall system coherence and aligning with business objectives.

Governance frameworks. Comprehensive approaches to managing agent lifecycle, ensuring ethical behavior, maintaining audit trails, and enabling human oversight. These frameworks must address unique challenges, such as AI explainability, bias detection, and accountability for autonomous decisions.

Quality and performance standards. Metrics and methodologies for evaluating agent performance, reliability, and business value contribution. These standards should enable consistent comparison across implementations and vendors.

An Architecture Development Methodology (ADM) for Agentic AI would provide systematic approaches for designing, implementing, and evolving agentic systems. This methodology would guide organizations through the complexities of agent selection, system integration, and operational management while ensuring alignment with business objectives and risk tolerance.

Our goal is not to create proprietary standards, but to contribute to open, industry-wide frameworks that accelerate enterprise adoption of Agentic AI. We believe that collaborative standardization efforts will benefit the entire ecosystem by reducing implementation risk, improving interoperability, and establishing clear value propositions for enterprise investment.

Let's join hands

The transformation from service-oriented to agent-oriented enterprise architectures represents one of the most significant shifts in enterprise computing since the advent of distributed systems. Success requires a coordinated effort across the industry, with vendors, enterprises, researchers, and standards organizations working together to establish robust and practical frameworks.

We invite the broader community to engage in this critical work. The terminology mapping presented here represents just the beginning of a much larger standardization effort. We need practitioners to validate and refine these concepts through real-world implementation. We need researchers to contribute theoretical frameworks and empirical studies. We need enterprises to share their requirements and constraints.

Most importantly, we need the industry to resist the temptation toward proprietary fragmentation that has hindered previous technology transitions. The potential of Agentic AI to transform enterprise operations is too significant to be constrained by incompatible vendor implementations and inconsistent architectural approaches.

Skan commits to open collaboration in this effort, sharing our insights from real-world implementations while learning from the broader community's expertise. Together, we can establish the architectural foundations that will enable Agentic AI to fulfill its transformative potential across the enterprise landscape.

The future of autonomous business process execution depends not only on advancing AI capabilities but also on creating standardized frameworks that make those capabilities accessible, reliable, and valuable to organizations of all sizes. The work begins now.